TP 3: Convolution et application des filtres

Dans le traitement d'image, un noyau ou un masque de convolution est une petite matrice utilisée pour effectuer des opérations sur les images, telles que, le lissage (atténuation du bruit), la détection des contours, etc. Ceci est accompli en faisant une convolution entre le noyau et une image. Ou plus simplement, lorsque chaque pixel de l'image de sortie est une fonction des pixels voisins (y compris lui-même) de l'image d'entrée et le noyau de cette fonction. L'opération de convolution est illustrée dans la Figure -1- .

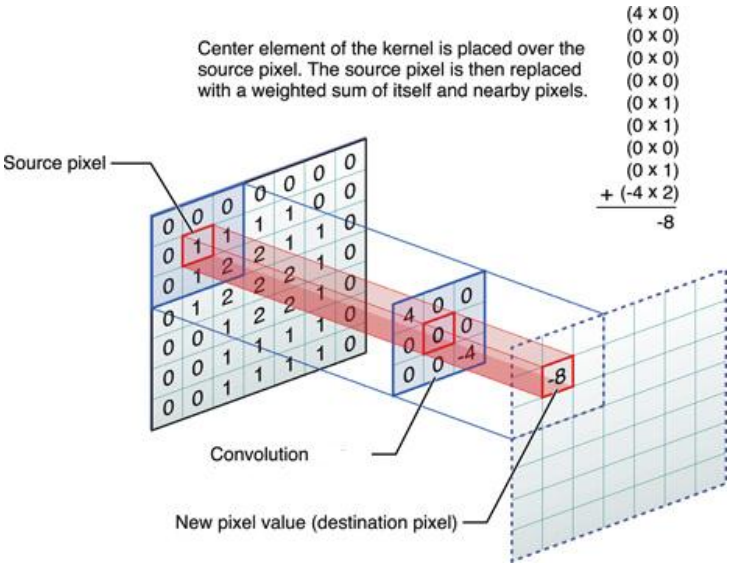

Figure 1 Convolution

1- Nous allons utiliser dans un premier temps la fonction prédéfinie **cv2.filter2D** d'OpenCV:

```
# Apply convolution using cv2.filter2D
img_conv = cv2.filter2D(src=gray_img, depth=cv2.CV_64F, kernel=kernel)
```

Écrire un programme qui lit une image en niveau de gris "gray_img", créer un noyau "kernel" et applique la fonction donnée ci-dessus.

Voici des exemples de noyau:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \times \frac{1}{30} \qquad \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

kernel 1 kernel 2

```
# Define the convolution kernels: blurring kernel
kernel1 = np.ones((5, 5))/30

kernel2 = np.array([[-1,-1,-1],[-1,8,-1],[-1,-1,-1]])
```

2- Nous allons à présent appliquer notre fonction de convolution (donnée ci-dessous) et comparer entre le résultat obtenu par les deux fonctions en les affichant dans la même fenêtre.

```
# Convolution function
def convolution(pad_img, kernel):
    p = int(kernel.shape[0]/2)
    pheight, pwidth = pad_img.shape
    img_conv = np.zeros(pad_img.shape)
    for i in range(p, pheight-p):
        for j in range(p, pwidth-p):
            roi = pad_img[i-p:i+p+1, j-p:j+p+1]
            img_conv[i, j] = np.sum(kernel * roi)
    img_conv = img_conv[p:-p, p:-p]
    return img_conv
```

Afin de pouvoir appliquer la fonction “convolution”, il faut gérer les pixels des contours de l’image (qui n’ont pas de voisins):

```
# Padding the image
p = int(kernel.shape[0]/2)
pad_img = np.zeros((gray_img.shape[0]+2*p, gray_img.shape[1]+2*p))
pad_img[p:-p, p:-p] = gray_img
```

3- Exercice :

Soit les noyaux suivants :

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \qquad \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \qquad \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix} \qquad \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \times \frac{1}{16}$$

Ecrire un programme qui permet de :

- Lire une image couleur et la convertir en niveau de gris (vous pouvez utiliser l'image « cablecar.bmp »)
- Diviser l'image en 16 blocs de même dimension, et appliquer sur chaque bloc un filtre dont le noyau est choisi d'une manière aléatoire parmi les noyaux ci-dessus
- Regrouper les 16 blocs filtrés dans une seule image et l'afficher.